Bugs in the original code of Pilotwings 64 (US)

As of 14 October 2026 (updated for stage 3: game patch "goal objects", data area, Expansion Pak). Based on a review of the whole decompilation (gcsmith/Pilotwings64Decomp, all files in src/kernel and src/app).

Rating

	Meaning
🔴 **Red**	Serious: crash, freeze, damaged memory or lost data. For section B, the game breaks as soon as a hack crosses the limit.
🟡 **Yellow**	Not that bad: wrong, but harmless or only a small inconvenience.
⚪ **Grey**	Only changes how the game plays, looks or sounds. **Not fixed on purpose**, so hacks feel like the original game. Can be ignored.

Checked: *verified* = I read the code myself and confirmed the finding. *Agent* = reported by a partial audit; I saw the code, but did not work out every consequence. A report about the glider models (`glider_toys.c`) was wrong and has been dropped.

Already fixed (code fixes and game patches in Hawk's Hangar)

Fix	Id	What it corrects
Safe text loading and drawing	safe-text-v1	The text loader runs over twice the length of each text. The line buffer of `uvFontPrintStr16` overflows (already at 43 characters plus a line break).
Room for 1024 texts and safer text handling	expanded-texts-v1	Text tables extended from 439 to 1024 entries (now at 0x801FE000 / 0x801FF000, in front of the program block). Missing text numbers return an empty text instead of NULL. Flight messages stop copying at the end of the text. ASCII strings are cut after 43 characters. Numbers that are too long for their place in a text are cut.
Correct saving of photos	photo-album-v1	Former A1–A3: photos with 5 or more objects lose data when saved; an empty photo keeps the old object count of its slot; resetting the album clears the wrong object list. Old saved games stay loadable.
Game patch: goal ring, ball target and hopper goals for every vehicle	goal-objects-v1	Not a code fix but a game patch (project settings, "Game patches", apply/undo; new projects get it). Fixes the three B entries "goal ring/ ball target set the vehicle to Rocket Belt" (task.c:358), "several Jumble Hopper goals read wrong data" (code_82520.c:93) and "hopper goal points always 0" (16-bit flags). See Mission_Scoring.md sections 7 and 9.1.

In the editor: checks for the number placeholders LEFT_CNT, STRACK, LEFT_SHT and C_POINTS (TextNumberSlots); custom texts can be added and removed, but only from number 439 on, so the numbers of the original texts never change.

A. Bugs in the unmodified game

Rating	#	Where	Bug	Effect	Checked
🟡	A13	balls.c:462 (ballsPopped)	After the search loop the wrong index is used; with 20 balls it points one entry behind the array.	Wrong bounce on freshly split balloons. Read access only.	Agent
○	A4	birdman2.c:618, rocket_belt.c:802	Typo `.y` instead of `.z` in the check whether the character is stuck on a surface. The help against getting stuck (US version only) therefore almost never works.	Birdman and Rocket Belt sometimes get stuck or jerk on surfaces.	verified
○	A5	code_82B90.c:2045 (Jumble Hopper, getting up after a fall)	Clamped to 0 instead of 90.	Visible jerk in the getting-up animation, short delay.	verified
○	A6	falco.c:182 (Meca Hawk)	Range `unk44 - unk38` instead of `unk44 - unk40`.	Wrong, possibly negative duration of a dive state of the AI.	verified
○	A7	geometry.c:117 (uvEndTmesh)	Triangle strips with more than 16 points lose 2 points at every split.	Gap in the thermal ring on the radar, missing triangles in effect trails. Visual only.	verified
○	A8	camera.c:843 (camera_802D532C)	Three variables are used before they are set.	Camera jerks or holds still briefly in camera modes 4 and 7.	Agent
○	A9	hang_glider1.c:135	The variable `sp5B` is never set (marked @bug in the decomp).	Input is rarely ignored for one frame.	verified
○	A10	env_sound.c:236	`sp26C` is only set on Crescent Island, but used on all maps.	Ambient sounds (ferry, planes, boats) partly do not follow their source.	verified

Rating	#	Where	Bug	Effect	Checked
○	A11	game.c:752	`sp50` is not set after a demo without a button press.	Whether the title intro plays after a demo is random.	Agent
○	A12	gyrocopter_sound.c:42	Sound curve filled one entry too far (marked @bug in the decomp).	Volume of one gyrocopter sound is off.	verified
○	A14	balls.c:385	The radar marker of a split balloon is placed at its velocity instead of its position.	Radar dot jumps for one frame.	Agent
○	A15	camera.c:592	The flag check `(x == 0) & 8` is always false.	Camera collision with the terrain always runs. That is the familiar feel of the game.	verified

B. Traps for romhacks

Harmless in the original, but dangerous as soon as the data changes. The editor has to check these limits, or a later code fix has to lift them.

Texts

Rating	Limit	Notes
🔴	At most 30 text pieces per frame.	More pieces overwrite memory. The text preview warns; a code fix is still missing.
🟡	Texts take heap memory every time they are loaded.	About 113 KB with 1024 short texts (measured in the emulator). `textLoadBlock` is called from 13 places.

File system and file formats

Rating	Limit	Notes
🔴	No new GZIP blocks in multi-entry files (UVEN, UVLV, UVTP, UVLT, UVTR, UVSQ).	When skipping a GZIP block the loader does not move on and the game hangs (filesystem.c). Our `--recompress` only repacks existing GZIP blocks, so it is safe.
🔴	Block size at most 153,599 bytes, unpacked and packed; always a multiple of 4.	Frame buffers serve as scratch buffers. A size that is not a multiple of 4 crashes the copy routine.
🔴	Maximum number of files per type: UVMD 400, UVCT 128, UVTX 500, UVEN 32, UVLT 4, UVTR 10, UVSQ 10, UVLV 150, UVAN 170, UVFT 20, UVBT 125, user files 128.	Not checked by the game.
🔴	At most 1000 ids per level (UVLV).	
🔴	Texture images at most 4096 bytes.	
🔴	Models used as moving objects: at most 24 parts.	
🔴	Fonts: at most 44 image blocks.	

Levels and missions

Rating	Limit	Notes
🔴	Level data (level.c): WOBJ 16, LPAD 14, TOYS 16, TPTS 16, APTS 20, BNUS 2.	Blocks are copied with the file size, without a check.
🔴	Mission data (task.c): 5000 bytes for all objects of a mission (sLevelObjHeap).	A ring takes 132 bytes. Block size and count in the COMM block must match. Enlarge this buffer before the mission editor, like the text tables (in the data area, in front of 0x801F5000).
🔴	Class, test and vehicle in the COMM block are only reported, not caught.	A Birdman class ⩾ 4 overwrites memory. At most 61 missions (fixed table).
🔴	Rings: at most 30, name at most 14 characters, at most 5 child rings, index below the ring count.	
🔴	Hover pads: at most 20, at most 5 follow-up pads, valid index.	
🔴	Landing pad type only 0 to 2.	
🔴	Balloons: at most 20.	
🔴	Map objects: at most 60 per test (rings, pads, targets and more together).	
🔴	Radar targets: at most 10 (HOPD + BTGT + used landing pads + runways).	
✅	Jumble Hopper with more than one goal: the wrong goal is read (`hopd++` plus `[i]`, code_82520.c:93).	**Fixed by the game patch goal-objects-v1** (the `hopd++` store at ROM 0x82818 becomes a nop).
✅	Jumble Hopper goal points are always 0 (also ball targets: 16-bit flags).	**Fixed by the game patch goal-objects-v1** (the module adds the two flag bits).
🔴	New tests: the fixed tables of the results screen (results.c) hold NULL for missing tests.	Crash. New tests need entries there.
🔴	Saved game: results are stored by number of tests, not by slot.	Gaps shift results.

Rating	Limit	Notes
🟡	Points are limited to 7 bits (at most 126).	
🟡	Terrain change (TPTS): unknown terrain numbers give a random value (task.c:713).	
✅	Rings and ball targets switch the music to Rocket Belt by assignment instead of comparison, and set the vehicle to Rocket Belt (task.c:358).	**Confirmed in the emulator: a Hang Glider at a goal ring crashes the game. Fixed by the game patch goal-objects-v1** (both stores become nops, the module plays the jingle of the real vehicle). Without the patch the editor warns before undoing it when a mission needs it.
🟡	Hang Glider and Birdman have the flags 0 until they land (flags are set only on landing or crash).	A flight that ends in the air (goal ring) scored nothing at all. The game module gives such a flight the flags 0xF7F8 (always, without patch button).
🟡	Medal limits for bonus games are read with different indices in different places.	Keep the values for bonus games equal.

Graphics and run time (rather for very large changes)

Rating	Limit	Notes
🔴	Terrain: at most 64 visible terrain tiles per frame, at most 1024 tiles per terrain (terrain.c, chan.c).	
🔴	Display list: 4200 commands per frame.	On overflow the game freezes (graphics.c).
🔴	Collision: at most 32 hits, partly exceeded (code_8170.c).	
🔴	Object pool: 100 moving objects (dobj).	Many places do not check whether a slot is free.
🔴	Paths and demos: camera paths at most 40 points (spath.c).	Demo and path files are copied with their file size into fixed buffers.

Memory (from memory.c)

Free heap of the original without Expansion Pak: about 1.8 MB, in three areas:

- 0x8004181C–0x800DA800

- 0x80125800–0x80200000 (since expanded-texts-v1 up to 0x801FE000, since stage 3 up to 0x801F5000)
- 0x803805E0–0x803DA800 (since the game module from 0x803905E0)

With an Expansion Pak it grows by 4 MB automatically (the heap ends at osMemSize). **Since stage 3 every hack requires the Expansion Pak**: without it the game module shows "EXPANSION PAK REQUIRED" and stops. Every hack ROM is 16 MiB. The current fill level is in `D_802B892C` (address 0x802B892C).

Areas the editor has taken from the heap:

RAM	Use	Since
0x801F5000–0x801FE000	**data area** (DataArea.cs): header + mission settings table (128 × 0x100 bytes, up to 0x801FD100), rest free	stage 3
0x801FE000–0x80200000	text tables (1024 names, 1024 texts)	expanded-texts-v1
0x803805E0–0x803905E0	**code area**: the game module	stage 2 (Step 28)
0x80080000 (150 KB, only without Expansion Pak)	frame buffer of the "Expansion Pak required" screen; the game never starts then	stage 3

Rules for the memory layout of future fixes:

- New **data** (tables, a larger mission pool) goes in front of the program block and grows downwards, below 0x801FE000 (today the data area starts at 0x801F5000; moving its start further down means changing the two instructions in uvMemInitBlocks and uvMemClearRegions, see DataArea.Patches).
- New **program code** goes behind the game's variables and grows upwards, from 0x803805E0. It is copied there at start-up from the end of the ROM (the game module, last 64 KiB of the ROM; the data area image lies in the 64 KiB in front of it).
- Always enlarge the big program block instead of adding small blocks. The memory manager only checks the start and the end of an allocation (func_8022A5F4), so a large allocation could cover a small block completely.
- The old text tables in the game's variables (0x80377F10–0x80378CCC, 3.4 KB) are unused since expanded-texts-v1 and can be reused.
- **Code cave** for small fixes: the body of uvMemScanBlocks (ROM 0x2B4A4–0x2B5A4, 0x100 bytes; the function only reports through the empty debug print). Used: 0x2B4A4–0x2B4D4 (photo-album-v1), 0x2B4D4–0x2B548 (loader and printf gate of the game module).

Suggested next code fixes

1. **Enlarge the mission object memory** (5000 bytes): needed for the mission editor (into the data area).
2. **Limit of 30 text pieces per frame**: the preview warns, but the game does not protect itself.